



ASSIGNMENT 3

SOFTWARE NOW

HIT 137

Summer Semester

2025

Group Name: SYDN 18

Group Members:

ASMITA PANTA - S398448

NASLA MAHARJAN - S398425

NAYANDEEP SHRESTHA - S397732

RAJAT SHRESTHA - S398548

Contents

Program.....	3
Collaboration Screenshots	17
Contributions	24
AI Declaration.....	25
References.....	26

A desktop application that demonstrates our understanding of Object-Oriented Programming principles, GUI development using Tkinter, and image processing using OpenCV.

Image_processing.py (Image processing in open cv , n.d.) (Linking openCV to a GUI, n.d.)

```
#importing libraries
import cv2 #open cv library for computer vision
import os

class ImageProcessor: #creating a class for image processing
    def __init__(self):
        #Initialize placeholder for the original image
        self.original_image = None
        self.image = None
        self.filename = None

    def load_image(self, path):
        #load the image from the file path and initializes the working copy
        self.original_image = cv2.imread(path) #imread return null if the path is
invalid
        self.image = self.original_image.copy() # creating a copy of original
image
        self.filepath = path
        self.filename = os.path.basename(path) # extracts base name from full
system path

    def reset_to_original(self): #discards all the edits and resotres original
image
        self.image = self.original_image.copy()

    def to_grayscale(self): #converts to gray scale
        gray = cv2.cvtColor(self.image, cv2.COLOR_BGR2GRAY)
        self.image = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

    def blur(self, intensity):
        #applies gaussian blur on user defined intensity
        if intensity > 0:
            k = intensity * 2 + 1 #k=kernal size must be odd number
            self.image = cv2.GaussianBlur(self.image, (k, k), 0)
```

```

def edge_detection(self):
    #Highlights outlines using canny edge detection
    gray = cv2.cvtColor(self.image, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 100, 200)
    self.image = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

def adjust_brightness(self, value):
    #shifts every pixel value up or down
    self.image = cv2.convertScaleAbs(self.image, beta=value)

def adjust_contrast(self, value):
    #multiplies pixel value to expand/shrink color range
    self.image = cv2.convertScaleAbs(self.image, alpha=value)

def rotate(self, angle):
    #Rotates the image by 90 180 270
    if angle == 90:
        self.image = cv2.rotate(self.image, cv2.ROTATE_90_CLOCKWISE)
    elif angle == 180:
        self.image = cv2.rotate(self.image, cv2.ROTATE_180)
    elif angle == 270:
        self.image = cv2.rotate(self.image, cv2.ROTATE_90_COUNTERCLOCKWISE)

def flip(self, mode):
    #flips the image horizontally or vertically
    if mode == "horizontal":
        self.image = cv2.flip(self.image, 1)
    else:
        self.image = cv2.flip(self.image, 0)

def resize(self, scale):
    #scales the image size
    height, width = self.image.shape[:2]

    #Calculate new dimensions
    new_width = int(width * scale)
    new_height = int(height * scale)

    #Select interpolation method
    if scale < 1:
        interpolation = cv2.INTER_AREA
    else:
        interpolation = cv2.INTER_CUBIC

```

```

#Resizing the image
self.image = cv2.resize(
    self.image,
    (new_width, new_height),
    interpolation=interpolation
)

```

Tkinter.py (Linking openCV to a GUI, n.d.) (Pyhton Tkinter, n.d.)

```

from Image_processing import ImageProcessor #imports ImageProcessor class from
the Image_processing file
from Image_History import ImageHistory ##imports ImageHistory class from the
Image_History file

#importing libraries
import tkinter as tk
from tkinter import filedialog, messagebox
import cv2
from PIL import Image, ImageTk

class ImageEditorApp:
    def __init__(self, root): # Initialize the main window and setup the
components
        self.root = root
        self.root.title("Image Processing Application")
        self.root.geometry("1000x600")
        #calls the logic classes
        self.processor = ImageProcessor()
        self.history = ImageHistory()
        #Build the visual interface
        self.create_menu()
        self.create_widgets()
        self.create_status_bar()

# ----- GUI SETUP ----- #
def create_menu(self):
    #Creates the top drop-down menus(File, Edit)
    menubar = tk.Menu(self.root)
    #adds open/save action
    file_menu = tk.Menu(menubar, tearoff=0)
    file_menu.add_command(label="Open", command=self.open_image)

```

```

file_menu.add_command(label="Save", command=self.save_image)
file_menu.add_command(label="Save As", command=self.save_as)
file_menu.add_separator()
file_menu.add_command(label="Exit", command=self.root.quit)
#add edit menu
edit_menu = tk.Menu(menuubar, tearoff=0)
edit_menu.add_command(label="Undo", command=self.undo)
edit_menu.add_command(label="Redo", command=self.redo)

menuubar.add_cascade(label="File", menu=file_menu)
menuubar.add_cascade(label="Edit", menu=edit_menu)
self.root.config(menu=menuubar)

def create_widgets(self):
    #Creates the main layout
    self.canvas = tk.Label(self.root)
    self.canvas.pack(side=tk.LEFT, padx=10, pady=10) #Left side image display
    #right side control pannel
    panel = tk.Frame(self.root)
    panel.pack(side=tk.RIGHT, fill=tk.Y)
    #adds buttons
    tk.Button(panel, text="Grayscale",
command=self.apply_grayscale).pack(fill=tk.X)
    tk.Button(panel, text="Edge Detection",
command=self.apply_edges).pack(fill=tk.X)

    # ----- Sliders (Non-Destructive) ----- #
    tk.Label(panel, text="Blur").pack()
    self.blur_slider = tk.Scale(panel, from_=0, to=10, orient=tk.HORIZONTAL,
command=self.preview_adjustments)
    self.blur_slider.pack(fill=tk.X)

    tk.Label(panel, text="Brightness").pack()
    self.brightness_slider = tk.Scale(panel, from_=-100, to=100,
orient=tk.HORIZONTAL, command=self.preview_adjustments)
    self.brightness_slider.pack(fill=tk.X)

    tk.Label(panel, text="Contrast").pack()
    self.contrast_slider = tk.Scale(panel, from_=1, to=3, resolution=0.1,
orient=tk.HORIZONTAL, command=self.preview_adjustments)
    self.contrast_slider.set(1)
    self.contrast_slider.pack(fill=tk.X)

```

```

        tk.Button(panel, text="Apply Adjustments",
command=self.commit_adjustments).pack(fill=tk.X, pady=5) #Makes the changes
permanent

        # ----- Transform Buttons ----- #
        tk.Button(panel, text="Rotate 90°", command=lambda:
self.rotate(90)).pack(fill=tk.X)
        tk.Button(panel, text="Rotate 180°", command=lambda:
self.rotate(180)).pack(fill=tk.X)
        tk.Button(panel, text="Rotate 270°", command=lambda:
self.rotate(270)).pack(fill=tk.X)

        tk.Button(panel, text="Flip Horizontal", command=lambda:
self.flip("horizontal")).pack(fill=tk.X)
        tk.Button(panel, text="Flip Vertical", command=lambda:
self.flip("vertical")).pack(fill=tk.X)

        tk.Button(panel, text="Resize 50%", command=lambda:
self.resize(0.5)).pack(fill=tk.X)
        tk.Button(panel, text="Resize 150%", command=lambda:
self.resize(1.5)).pack(fill=tk.X)

    def create_status_bar(self):
        self.status = tk.Label(self.root, text="No image loaded",
relief=tk.SUNKEN, anchor=tk.W)
        self.status.pack(side=tk.BOTTOM, fill=tk.X)

    # ----- CORE LOGIC ----- #
    def update_display(self):
        #converts the opencv image(BGR) to Tkinter format(RGB)
        image = cv2.cvtColor(self.processor.image, cv2.COLOR_BGR2RGB)
        image = Image.fromarray(image)
        #fit the image to screen wihtout stretching
        image.thumbnail((600, 500))
        self.tk_image = ImageTk.PhotoImage(image)
        self.canvas.config(image=self.tk_image)
        #update the text at the bottom
        h, w = self.processor.image.shape[:2]
        self.status.config(text=f"{self.processor.filename} | {w} x {h}")

    def reset_sliders(self):
        #resets slider positions to default
        self.blur_slider.set(0)
        self.brightness_slider.set(0)

```

```

        self.contrast_slider.set(1)

# ----- FILE OPERATIONS ----- #
def open_image(self):
    #opens a file dialog and loads the selected image into the gui
    path = filedialog.askopenfilename(filetypes=[("Images",
    "*.png;*.jpg;*.jpeg;*.bmp")])
    if path:
        self.processor.load_image(path)
        self.history.save(self.processor.image)
        self.update_display()
        self.reset_sliders()

def save_image(self):
    #saves the edited file (overwrites the current verison)
    if self.processor.image is not None:
        cv2.imwrite(self.processor.filepath, self.processor.image)
        messagebox.showinfo("Save Image", "Image saved successfully!")
    else:
        messagebox.showwarning("Save Image", "No image to save.")

def save_as(self):
    #saves as new file (does not overwrite the current version)
    path = filedialog.asksaveasfilename(defaultextension=".png",
    filetypes=[("PNG files", "*.png"), ("JPEG files", "*.jpg;*.jpeg"), ("BMP files",
    "*.bmp")])
    if path:
        cv2.imwrite(path, self.processor.image)
        messagebox.showinfo("Save Image", "Image saved successfully!")

# ----- EDIT OPERATIONS ----- #
def undo(self):
    #reverts to previous image version
    self.processor.image = self.history.undo(self.processor.image)
    self.processor.original_image = self.processor.image.copy()
    self.update_display()
    self.reset_sliders()

def redo(self):
    #repeats the process again
    self.processor.image = self.history.redo(self.processor.image)
    self.processor.original_image = self.processor.image.copy()
    self.update_display()
    self.reset_sliders()

```



```

# ----- IMAGE TRANSFORMATIONS ----- #
def apply_grayscale(self):
    self.history.save(self.processor.image)
    self.processor.to_grayscale()
    self.processor.original_image = self.processor.image.copy()
    self.update_display()

def apply_edges(self):
    self.history.save(self.processor.image)
    self.processor.edge_detection()
    self.processor.original_image = self.processor.image.copy()
    self.update_display()

def rotate(self, angle):
    self.history.save(self.processor.image)
    self.processor.rotate(angle)
    self.processor.original_image = self.processor.image.copy()
    self.update_display()

def flip(self, mode):
    self.history.save(self.processor.image)
    self.processor.flip(mode)
    self.processor.original_image = self.processor.image.copy()
    self.update_display()

def resize(self, scale):
    self.history.save(self.processor.image)
    self.processor.resize(scale)
    self.processor.original_image = self.processor.image.copy()
    self.update_display()

# ----- Slider Operations ----- #
def preview_adjustments(self, _):
    #Updates the image in real-time as sliders move.
    self.processor.reset_to_original()
    self.processor.blur(self.blur_slider.get())
    self.processor.adjust_brightness(self.brightness_slider.get())
    self.processor.adjust_contrast(self.contrast_slider.get())
    self.update_display()

def commit_adjustments(self):
    #Finalizes the slider changes so they can be Undone/Redone.
    self.history.save(self.processor.image)

```

```

        self.processor.original_image = self.processor.image.copy()
        self.update_display()
        messagebox.showinfo("Adjustments Applied", "Adjustments have been applied
to the image.")

# =====
# Program Entry Point
# =====
if __name__ == "__main__":
    root = tk.Tk()
    app = ImageEditorApp(root)
    root.mainloop()

```

Image_History.py

```

class ImageHistory:
    def __init__(self):
        #initializes placeholder
        self.undoStack = []
        self.redoStack = []

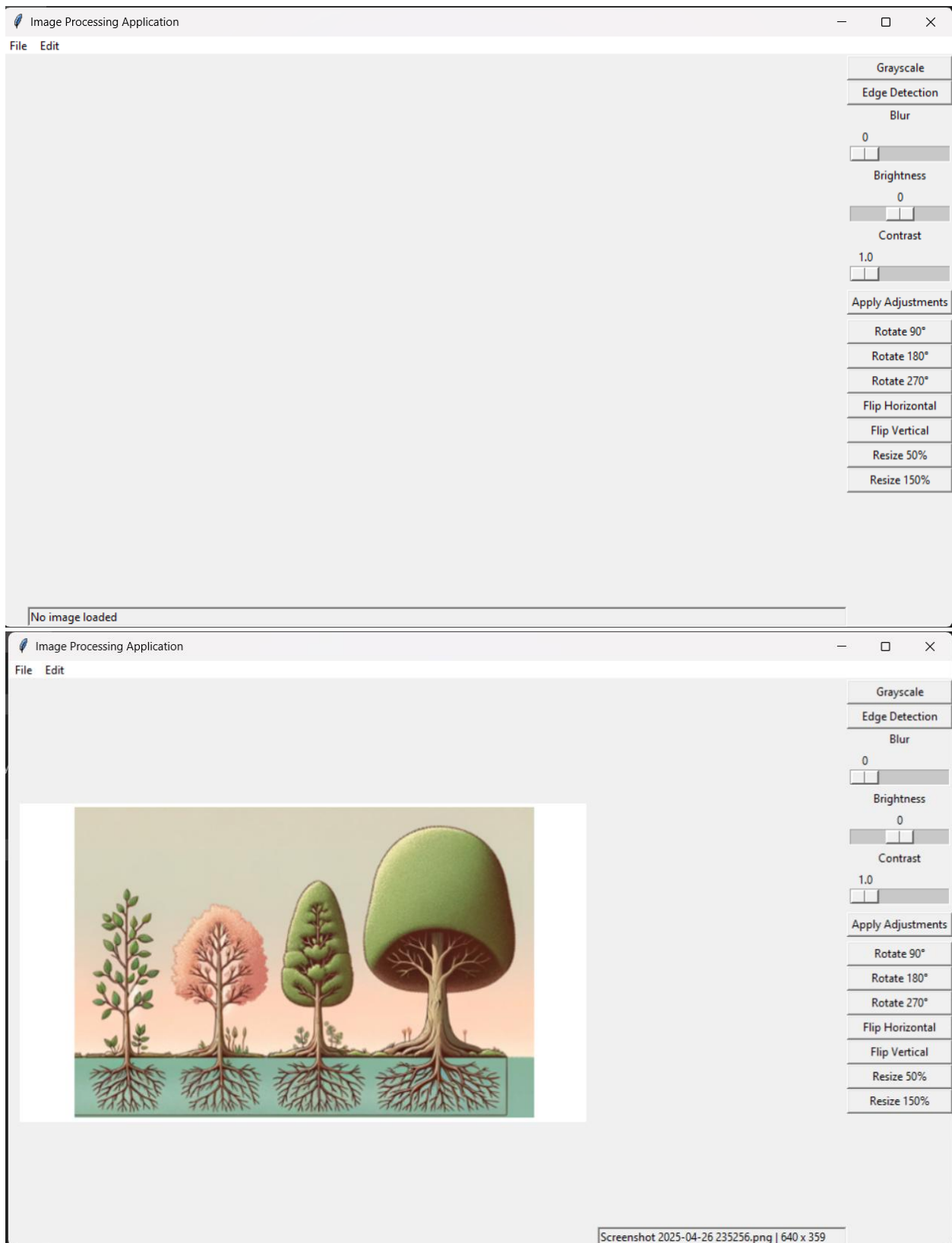
    def save(self, image):
        #saves the image
        self.undoStack.append(image.copy())
        self.redoStack.clear()

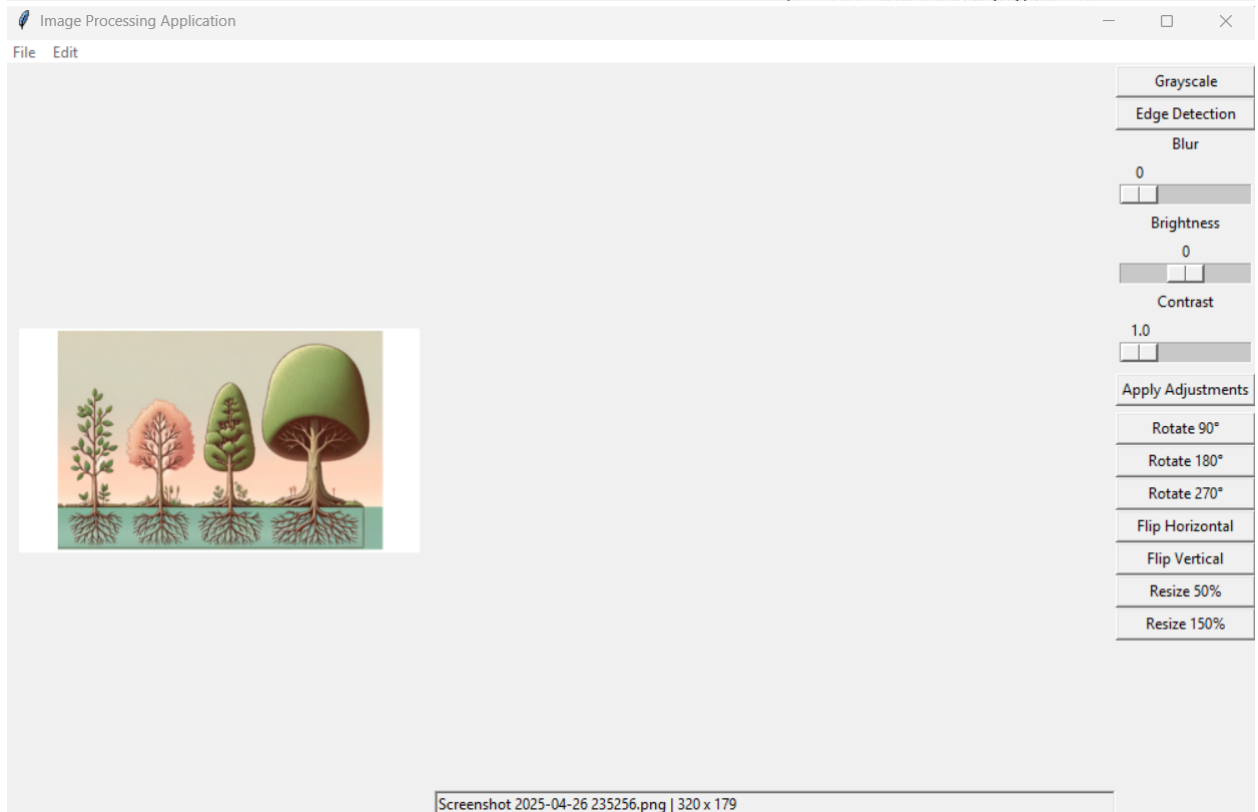
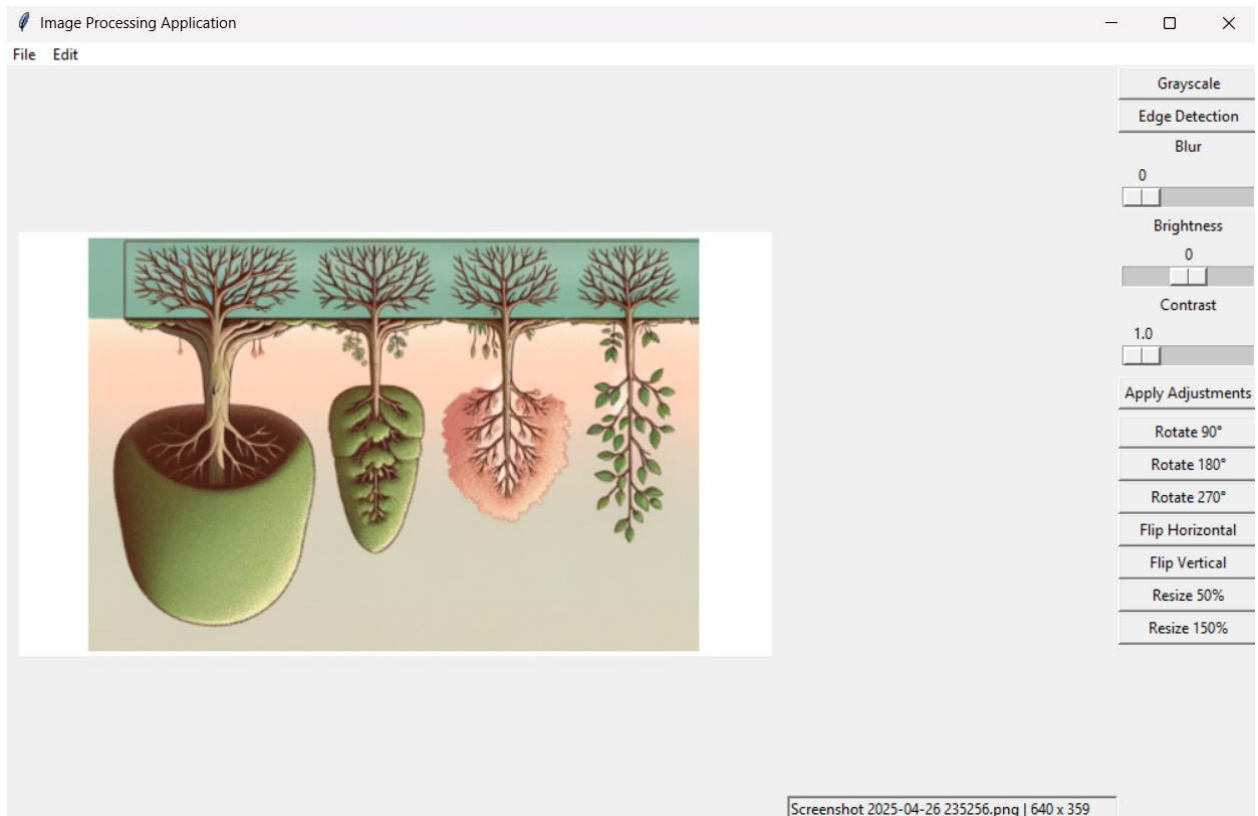
    def undo(self, current):
        #it reverts back to the previous condition
        if self.undoStack:
            self.redoStack.append(current.copy())
            return self.undoStack.pop()
        return current

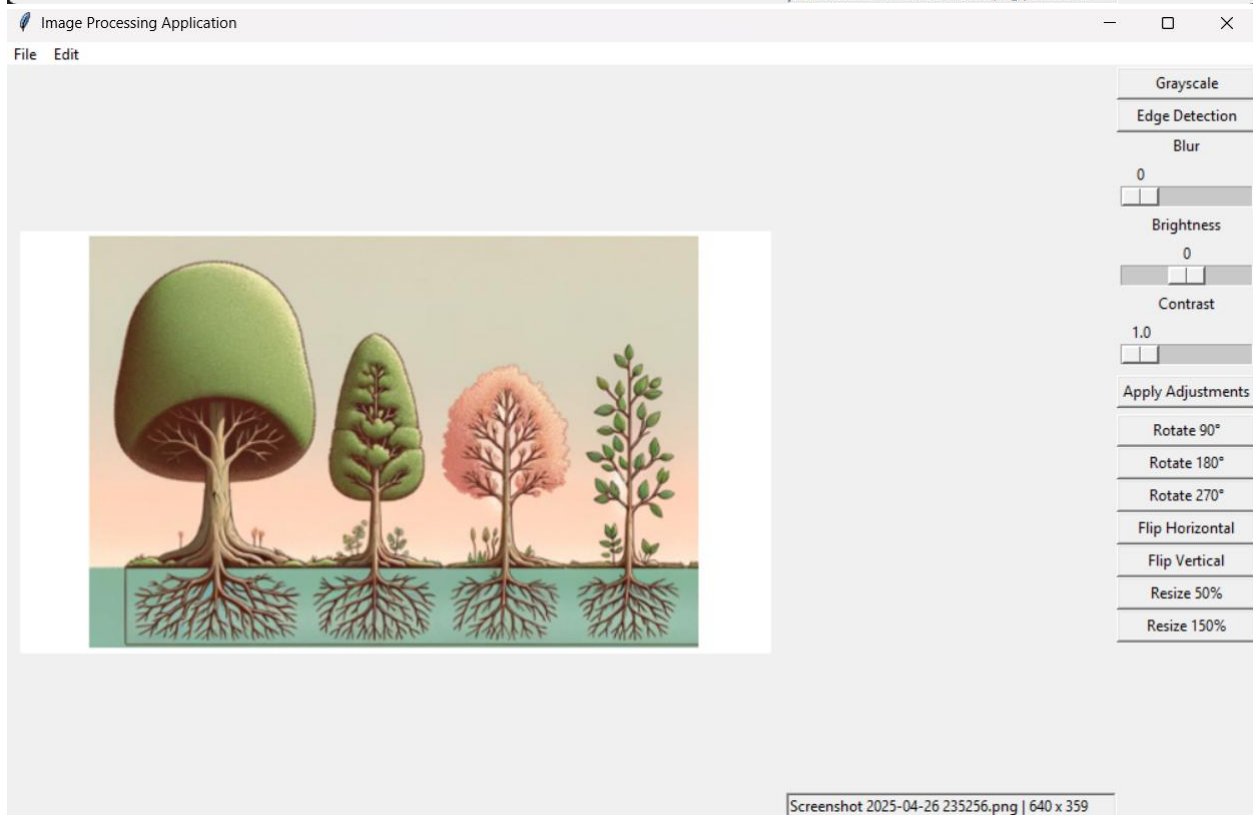
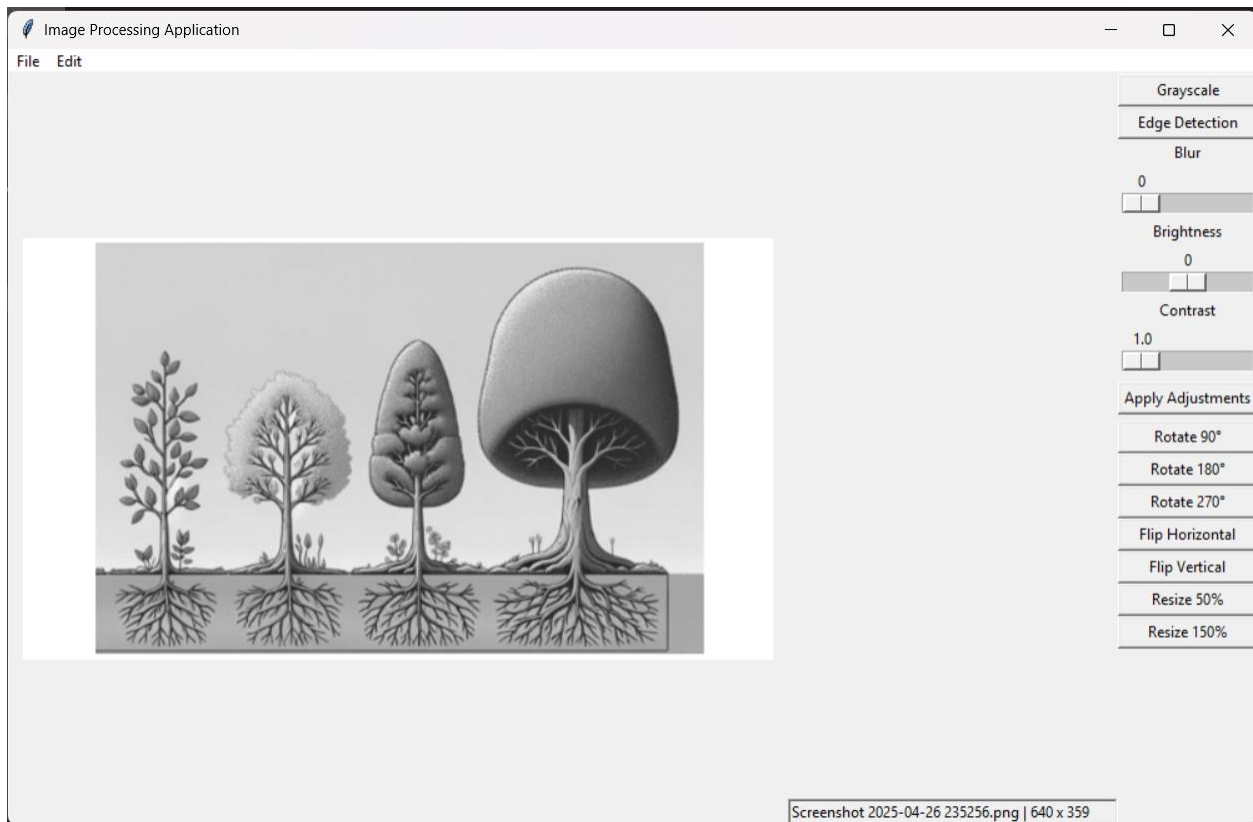
    def redo(self, current):
        #applies the same funtion again
        if self.redoStack:
            self.undoStack.append(current.copy())
            return self.redoStack.pop()
        return current

```


OUTPUTS:











Collaboration Screenshots

GitHub collaboration:

January 29, 2026 – February 5, 2026

Period: 1 week ▾

Overview

1 Active pull requests

0 Active issues

1

Merged pull requests

0

Open pull requests

0

Closed issues

0

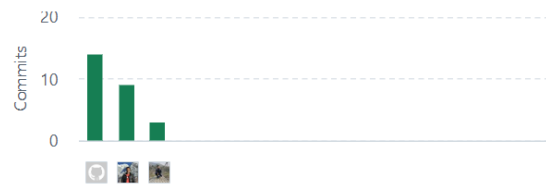
New issues

Summary

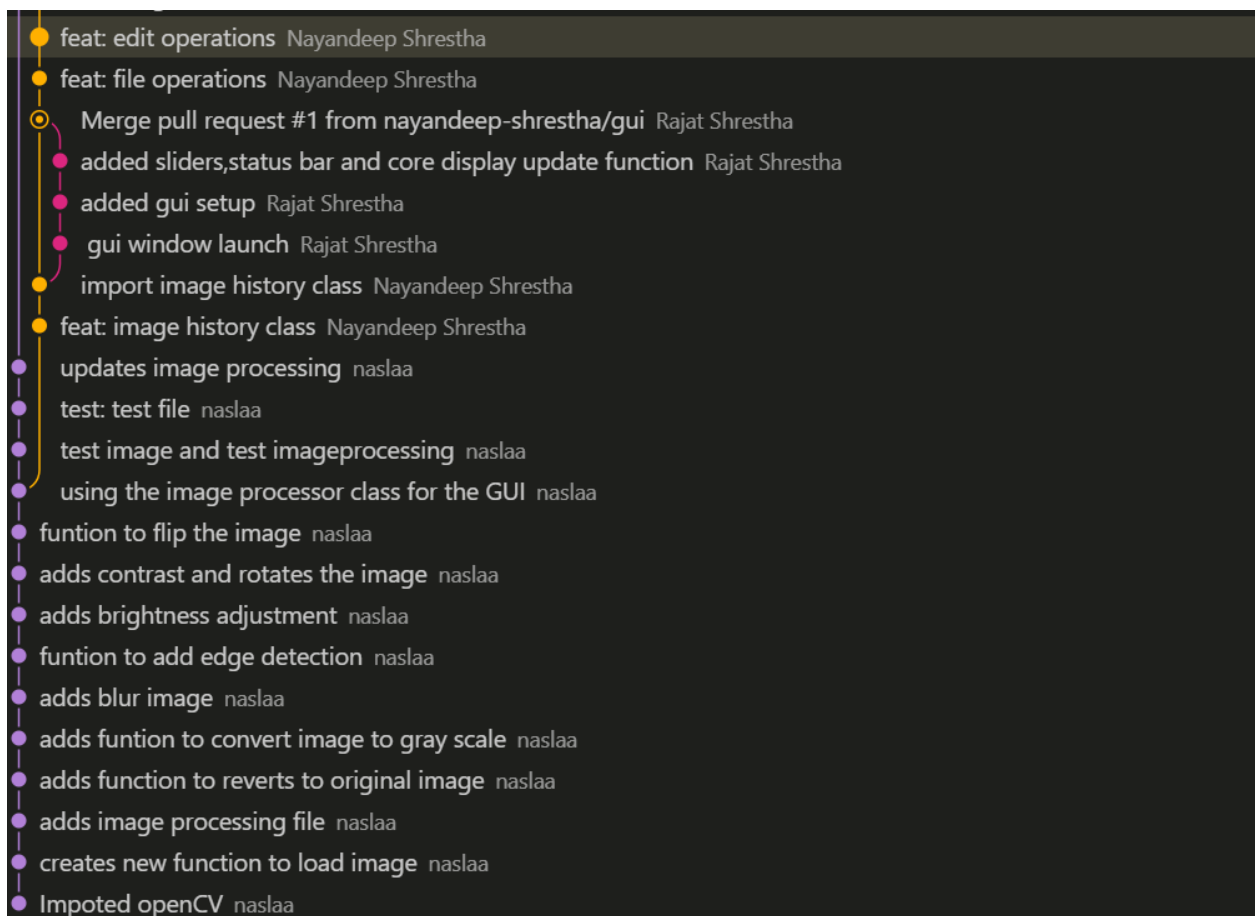
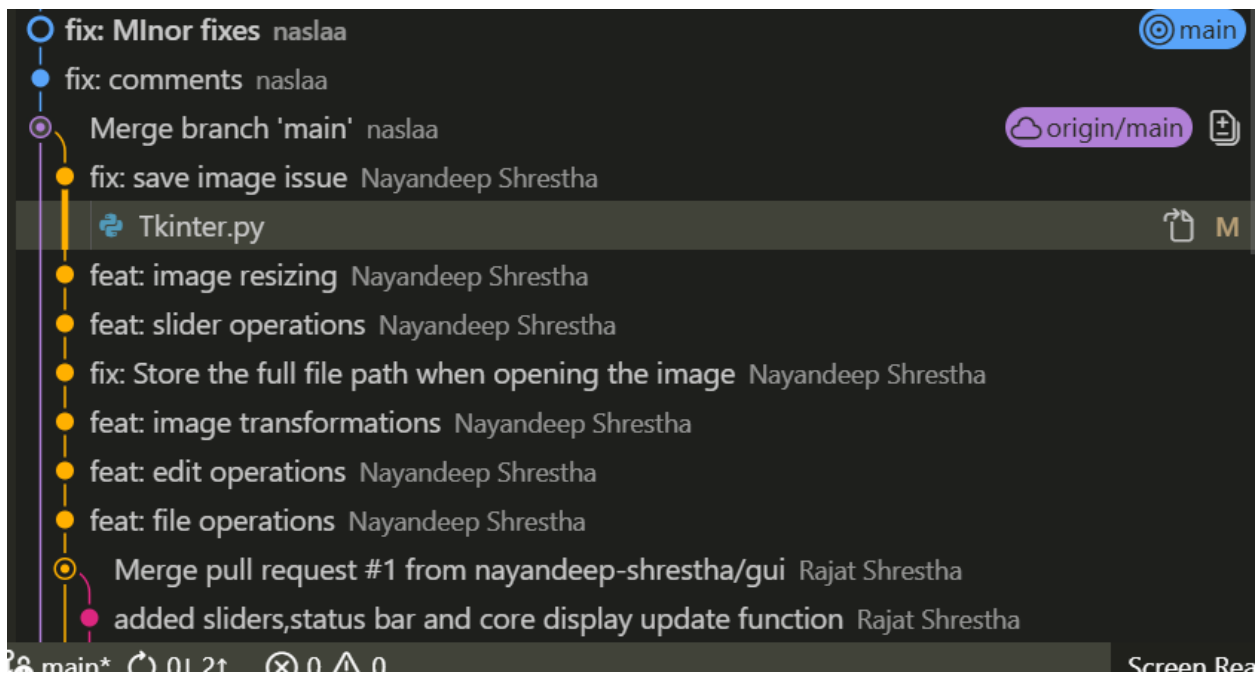
Excluding merges, **3 authors** have pushed **26 commits** to main and **26 commits** to all branches.

On main, **0 files** have changed and there have been **0 additions** and **0 deletions**

Top Committers



1 pull request merged by 1 person



Teams collaboration:

**HIT137_2025_SUM_SE...** Chat +1 

Today

 5:52 PM

Nayandeep Shrestha Rajat Shrestha assignment
Please create a tkinter web for image processing

Nayandeep Shrestha 6:01 PM

NS Okay sure

Rajat Shrestha 7:25 PM

RS i have completed the gui now file operation is left

Nayandeep Shrestha 7:25 PM

NS I'll be doing that

Rajat Shrestha 7:26 PM

RS okay

Nayandeep Shrestha 8:25 PM

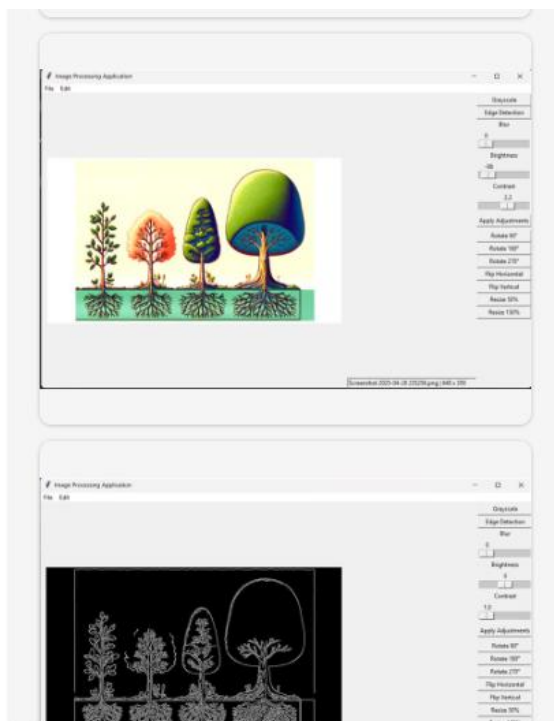
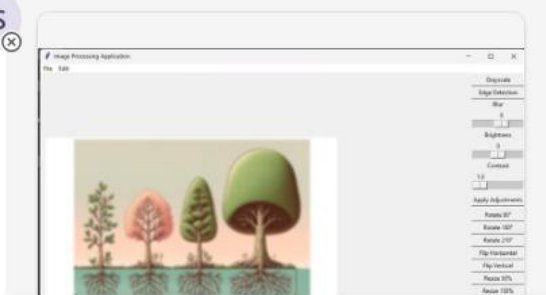
NS guys I have done the remaining part

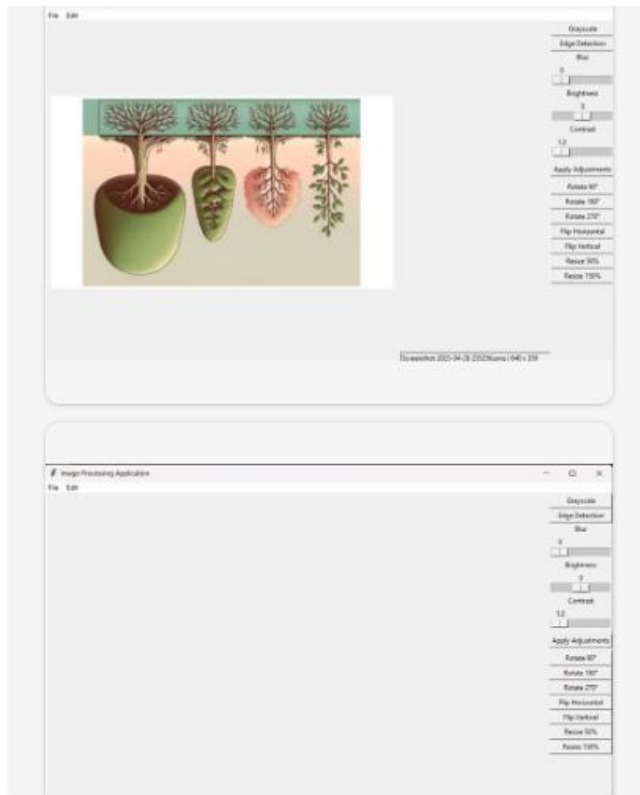
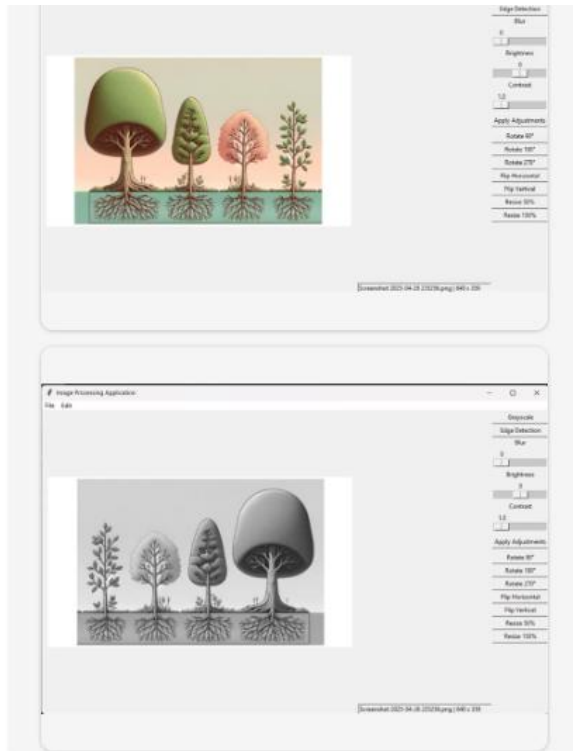
8:25 PM

okay i'll do my remaining

Nayandeep Shrestha 8:26 PM

NS



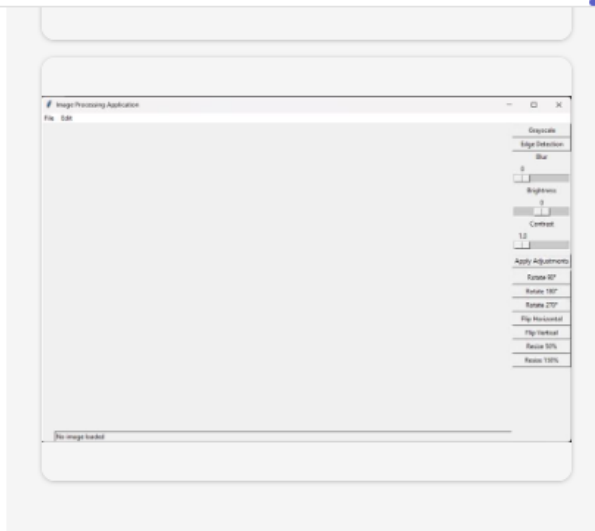




HIT137_2025_SUM_SE...



Chat +1



documentation is remaining

Nasla Maharjan 2/5/2026 8:25 PM
okay i'll do my remaining

Contributions

1. Asmita Panta - S398448

- Arranged all the citations in the document.
- Added collaboration screenshots and documented contributions.

2. Nasla Maharjan- S398425

- Created an image processing file.
- Tested image processing in a Jupyter file.
- Implemented an image processing class that implements all the functions given in the requirement.
- Pushed the code to GitHub for collaboration.

3. Nayandeep Shrestha - S397732

- Created an image history file.
- Created a GUI using Tkinter that implements all the functions given in the requirement.
- Tested image in Tkinter GUI.
- Pushed the code to GitHub for collaboration.

4. Rajat Shrestha- S398548

- Created Gui Window launch
- Add Gui setup
- Added sliders, status bar and core display update function
- Pushed the code to Github for collaboration

AI Declaration

This assignment was done on our own with the help of the listed references. Any kind of AI tools or LLM has not been used for the content generation of this assignment.

References

Image processing in open cv . (n.d.). (opencv) Retrieved 2 2026, from
https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html

Linking openCV to a GUI. (n.d.). (OpenCv forum) Retrieved 2 2026, from
<https://forum.opencv.org/t/linking-opencv-to-a-gui/7388>

Pyhton Tkinter. (n.d.). (GeeksforGeeks) Retrieved 2 2026, from
<https://www.geeksforgeeks.org/python/python-gui-tkinter/>